

Statistical Inference and Modeling

2102575

Suwichaya Suwanwimolkul¹

¹Electrical Engineering
Chulalongkorn University

Semester II, 2025

- Lecture 1: Introduction
- Lecture 2: Model selection
- Lecture 3: Resampling
- Lecture 4: Linear regression
- Lecture 5: Robust regression
- Lecture 6: Classification
- Lecture 7: Logistic regression
- Lecture 8: Neural networks
- Lecture 9: K-nearest neighbor
- Lecture 10: Bayesian decision, LDA, QDA, and naive Bayes
- Lecture 11: Support vector machine
- Lecture 12: Tree-based methods
- Lecture 13: Bagging, random forest, boosting
- Lecture 14: Principal component analysis
- Lecture 15: Clustering algorithms
- Lecture 16: Gaussian mixture model clustering
- Lecture 17: DBSCAN clustering

14. Principal component analysis

2102575

Suwichaya Suwanwimolkul¹

¹Electrical Engineering
Chulalongkorn University

Semester II, 2024

- 1 Table of contents
- 2 Supervised vs unsupervised learning
- 3 Principal component analysis (PCA)
 - The idea
 - What is a good projection matrix?
 - The solutions are the eigenvectors

The minimum-error formulation

- 4 PCA applications
 - data visualization (with codes)
 - incomplete data observation
- 5 Appendix

Supervised vs unsupervised learning

Supervised learning

$$\{X_j, y_j\}_{j=1}^m$$

Using $X_j \rightarrow \hat{y}_j$

Unsupervised learning

$$\{X_j\}_{j=1}^m$$

Using $X_j \rightarrow$ the data behavior

Examples of unsupervised learning

- ① data clustering
- ② dimensionality reduction: *→ Today*
 - data visualization
 - data compression
- ③ data association (rule-based machine learning)

Characteristics of unsupervised learning

- ① No ground truth $\rightarrow$ hard to assess the result from the model.
- ② More subjective $\rightarrow$ no simple goal for the analysis $\rightarrow$ usually based on rules.

Unsupervised learning algorithms

- ① Principle component analysis (PCA) ←
- ② K-mean clustering
- ③ hierachical clustering
- ④ GMM
- ⑤ DBSCAN

- 1 Table of contents
- 2 Supervised vs unsupervised learning
- 3 **Principal component analysis (PCA)**
 - The idea
 - What is a good projection matrix?
 - The solutions are the eigenvectors

The minimum-error formulation

- 4 PCA applications
 - data visualization (with codes)
 - incomplete data observation
- 5 Appendix

PCA: the idea

- Let x_j be an input features where the dimension of the features is n . Here the subscription j denotes the index of the data points.
- Principal component analysis (PCA) is about finding the projection matrix Φ that can reduce the dimension of input features from n to p where $p \leq n$.

$$z_j = Px_j$$

$$\begin{aligned} \bar{z} &= P x \\ \begin{bmatrix} \bar{z} \end{bmatrix}_{2 \times 1} &= \begin{bmatrix} P \end{bmatrix}_{2 \times 3} \begin{bmatrix} x \end{bmatrix}_{3 \times 1} \end{aligned}$$

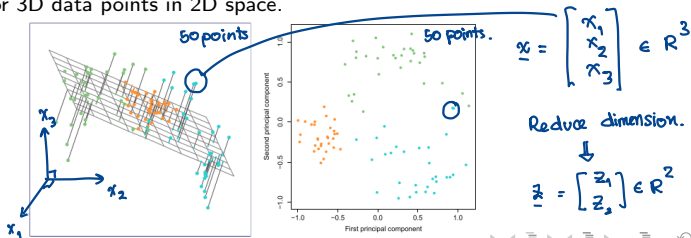
where $P \in \mathbb{R}^{p \times n}$, and p is the resulting dimension after project.

PCA is often used for dimensionality reduction and data visualization. Therefore, it is often that p is a small value (e.g., $p = 1$ or 2 for visualization). The resulting vector z_j is a vector in a lower dimensional space (p -dimensional space).

P maps an n dim space to a p dim space $p \leq n$

Example:

One can use PCA to enable the visualization of data in lower dimension, e.g., visualizing the 2D data with a 1D axis, or 3D data points in 2D space.



PCA: what is a good projection matrix?

* คำถามที่ว่า Q^T ดีแค่ไหน : Q^T map จาก $\underline{x} \in \mathbb{R}^2$ ไปยัง $\underline{z} \in \mathbb{R}^1$

Let $\mathbf{X}$ be a matrix of m data points where each point is in 2D, i.e., $\mathbf{x}_j \in \mathbb{R}^2$, and each data point is standardized, i.e., $E[\mathbf{x}_j] = \mu = 0$ and $E[(\mathbf{x}_j - \mu)^2] = \|\mathbf{x}_j\|^2 = 1$. $\rightarrow$ Standard Scaler

$$\mathbf{X} := [\mathbf{x}_1 \quad \mathbf{x}_2 \quad \dots \quad \mathbf{x}_j \quad \dots \quad \mathbf{x}_m] = \begin{bmatrix} x_{1,1} & x_{2,1} & \dots & x_{j,1} & \dots & x_{m,1} \\ x_{1,2} & x_{2,2} & \dots & x_{j,2} & \dots & x_{m,2} \end{bmatrix} = \begin{bmatrix} x_{:,1}^T \\ x_{:,2}^T \end{bmatrix}$$

Target: we want to reduce from 2D to 1D using a matrix $\mathbf{P} = \mathbf{Q}^T$: *

$$\mathbf{z}_j = \underset{1 \times 2}{\mathbf{Q}^T} \mathbf{x}_j = [\mathbf{q}_1^T] \mathbf{x}_j = \underbrace{[q_{1,1} \quad q_{1,2}]}_{1 \times 2} \begin{bmatrix} x_{j,1} \\ x_{j,2} \end{bmatrix} = \underline{[z_{j,1}]}$$

Let us apply the above projection to the entire $\mathbf{X}$

$$\mathbf{Z} = \mathbf{Q}^T \mathbf{X} = [\mathbf{q}_1^T] \mathbf{X} = [\mathbf{q}_1^T] \begin{bmatrix} x_{1,1} & x_{2,1} & \dots & x_{j,1} & \dots & x_{m,1} \\ x_{1,2} & x_{2,2} & \dots & x_{j,2} & \dots & x_{m,2} \end{bmatrix} = \begin{bmatrix} \underline{z_1} & \dots & \underline{z_m} \end{bmatrix}$$

Note that the size of $\mathbf{Z}$ is $1 \times m$

PCA: let's reduce the above problem to just one projection vector

What is a good q_1 [1]?

- A good q_1 should capture most of the information of the data.
 $\Rightarrow$ This translates into q_1 that maximizes the variance of the projected data (Z).

$$\Rightarrow \text{maximize } ZZ^T = q_1^T XX^T q_1$$

- While Z is being maximized, we also need to ensure the resulting ZZ^T will not go to infinity. So, a good q_1 should prevent this from happening. This is done by adding the constraint:

$$\Rightarrow \underbrace{q_1^T q_1}_{=1} = 1$$

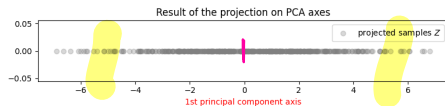
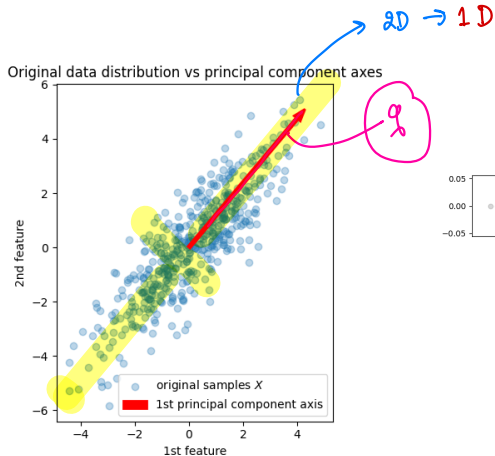
Put the two requirements together; one can solve for q_1 (the first principal component: the 1st PCA) via

$$\operatorname{argmax}_{q_1} q_1^T XX^T q_1 \tag{1a}$$

$$\text{subject to } q_1^T q_1 = 1 \tag{1b}$$

What does this look like?

PCA: let's reduce the above problem to just one projection vector



Code: [PCA tutorial](#)

PCA: the second projection vector

The second projection vector (2nd PCA) can be found through solving Eq.(1), similar to the 1st PCA. However, it is best to make sure that the 2nd PCA will not be the same as the first one, which leads to the following formulation:

$$\operatorname{argmax}_{q_2} q_2^T X X^T q_2 \quad (2a)$$

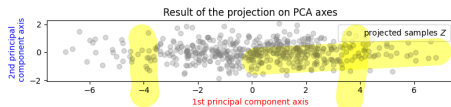
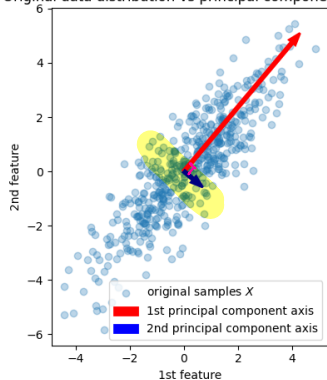
$$\text{subject to } q_2^T q_2 = 1 \quad (2b)$$

$$q_2^T q_1 = 0 \quad (2c)$$

- What does this look like?
- q_1 and q_2 are orthogonal ...

PCA: two projection vectors

Original data distribution vs principal component axes



Code: [PCA tutorial](#)

Therefore, one need to solve the following problems to find the projection matrix $Q^T = \begin{bmatrix} q_1^T \\ q_2^T \end{bmatrix}$.

- At first, we find q_1 by solving

$$\operatorname{argmax}_{q_1} q_1^T X X^T q_1$$

$$\text{subject to } q_1^T q_1 = 1$$

- Once we obtain q_1 , we will find q_2 by solving

$$\operatorname{argmax}_{q_2} q_2^T X X^T q_2$$

$$\text{subject to } q_2^T q_2 = 1$$

$$q_2^T q_1 = 0$$

$$\operatorname{argmax}_{q_3} q_3^T X X^T q_3$$

$$\text{s.t. } q_3^T q_2 = 1$$

$$q_3^T q_2 = 0$$

$$q_3^T q_1 = 0$$

Let's see how to use q_1 and q_2 ...

Then, we will show that in fact, q_1 and q_2 are indeed the eigen vectors of XX^T .

PCA: the solution for q_1 and q_2 have to be eigenvectors

Let us explore the reason why q (both q_1 and q_2) is an eigen vector of XX^T :

$$\underset{q}{\operatorname{argmax}} q^T XX^T q$$

$$\text{subject to } q^T q = 1$$

You can simplify the problem using Lagrangian: let α be a Lagrange multiplier.

$$\underset{\alpha; \alpha > 0}{\operatorname{argmax}} \sup_{q} \mathcal{L}(q, \alpha) = (q^T XX^T q) - \alpha(q^T q - 1)$$

q that gives the supremum (stationary points) is the point that causes $\nabla_q \mathcal{L}(q, \alpha) = 0$

$$\nabla_q \mathcal{L}(q, \alpha) = \underbrace{XX^T q - \alpha q}_{\text{stationary}} = 0 \Rightarrow \underbrace{XX^T q = \alpha q}_{\text{eigenvalue}}$$

The optimal q has to satisfy

$$XX^T q = \alpha q \quad \text{or} \quad (XX^T - \alpha I)q = 0$$

This show that q has to be an eigenvector, which is a member of a fixed set of vectors. Note that q that gives the highest value is **the eigenvector with the largest eigenvalue**.

Let us replace $XX^T q$ with αq in the primal problem.

Then, the objective function will be

$$q^T XX^T q = \alpha q^T q$$

PCA: notes

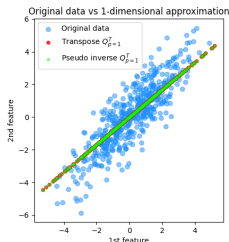
- Usually, the first principal component is the eigenvector with the largest eigenvalue, (and α is the largest eigenvalue).
- This means that all the solutions (q) will be orthogonal to each other. → Thus, the transformation matrix in PCA is an orthogonal matrix.
- Suppose that X is an n -dimensional feature vector. Immediately, we can compute the transformation matrix $Q \in \mathcal{R}^{n \times n}$ from the Eigen-decomposition of XX^T .
- Then, we can pick p eigen vectors (associated with the first p largest eigenvalues) to provide the transformation matrix in PCAs, resulting into z_j with p -dimensional data. → for mapping

$$z = Q_p^T x \quad \text{where } Q_p = [q_1 \quad \dots \quad q_p]$$

* from n to p

- This means that we can approximate x_j from z_j using the following relationship:

$$x \approx Q_p z \dots \text{ or } x \approx \text{pseudo inverse}(Q_p^T) z$$



PCA: the minimum-error formulation *

Previously: we want to reduce from n to p dimensions using a transformation matrix Q_p^T :

$$z_j = Q_p^T x_j$$

Alternative formulation: But this time, we consider a new direction (by projecting from 2D to 3D) using a matrix Ω . One can use Ω = pseudo inverse of (Q_p^T) , or use the property of the orthogonal matrix :

$$x_j \approx \Omega z_j$$

$$\Omega = \text{pseudo inverse of } (Q_p^T) \quad \text{or} \quad \text{where } \Omega = Q_p = [q_1 \quad q_2 \quad \dots \quad q_p]$$

One can solve for Ω via

Matrix completion

$$\underset{\Omega \in \mathbb{R}^{n \times p}, z_j \in \mathbb{R}^p \quad \forall j=1, \dots, m}{\operatorname{argmin}} \sum_{j=1}^m \|x_j - \Omega z_j\|^2$$

Diagram:
 $x \in \mathbb{R}^n \xrightarrow{Q^T} \tilde{x} \in \mathbb{R}^p$
 $\tilde{x} \in \mathbb{R}^p \xrightarrow{\Omega} x \in \mathbb{R}^n$
 $P = Q^T$
 $\Omega = \text{inv}(Q^T) = P$

This optimization problem has two sets of unknown variables $\Omega \in \mathbb{R}^{n \times p}$, $z_j \in \mathbb{R}^p$ for all $j = 1, \dots, m$. Here, we can put all z_j together as a matrix Z , where $Z := [z_1, \dots, z_m]$.

- * We can solve for Ω by ... (1) solve for Q using the previous PCA technique and ... (2) compute Ω (e.g., by Ω = pseudo inverse of (Q_p^T) or $\Omega = Q_p$).
- When $p < n$, the first p principal components provide the best p -dimensional approximation to the original data.
- When $p = \min(m - 1, n)$, these principal components provide the exact representation of the original data.
*→ no missing features in $\mathbb{R}^n$ ****
- This setting has a complete data observation as we observe all x_j for $j = 1, 2, \dots, m$.

- 1 Table of contents
- 2 Supervised vs unsupervised learning
- 3 Principal component analysis (PCA)
 - The idea
 - What is a good projection matrix?
 - The solutions are the eigenvectors

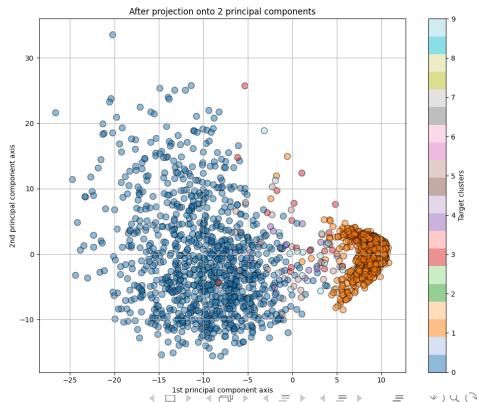
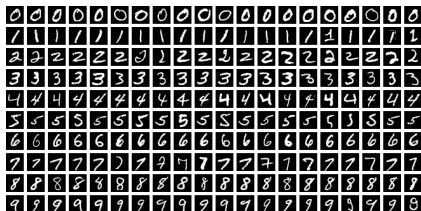
The minimum-error formulation

- 4 **PCA applications**
 - data visualization (with codes)**
 - incomplete data observation**
- 5 Appendix

PCA for data visualization (with codes)

Let's visualize the clustering of a set of MNIST images [3] using PCA. The image is of size 28×28 . To see how these images are associated to one another, we see the clustering of these images by reducing the entire image dimensions ($28 \times 28 = 784$) to 2 dimensions. The steps are as follows (code is here: [HW5](#)):

- 1 Read the data.
- 2 Before we apply the PCA, we need to standardize our data.
- 3 Apply PCA.
- 4 Visualize the result projected data.



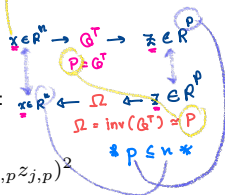
PCA for incomplete data observation * $x_i \in \mathbb{R}^n$ but missing some values in the feature dimension.

Missing value can be recovered by projecting data from the principal components to the higher dimension.

$$x = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} \leftarrow z = \begin{bmatrix} z_1 \\ z_2 \\ \vdots \\ z_p \end{bmatrix}$$

Principal Components with Missing Values

- The missing values can be estimated using the following optimization:



Previously:

$$\underset{\Omega \in \mathbb{R}^{n \times p}, Z \in \mathbb{R}^{p \times m}}{\operatorname{argmin}} \sum_{(i,j) \in \mathcal{O}} (x_{i,j} - \sum_p \Omega_{i,p} z_{j,p})^2$$

where $\mathcal{O}$ is the set of observed pairs of indices (i,j) , a subset of the possible $n \times m$ pairs.

- You should observe the similarity/difference between this optimization problem and the previous one.
- Solving the optimization problem directly is difficult (we partially observe $(i,j) \in \mathcal{O}$, unlike the complete data observation).
- The iterative algorithm presented in next slide is often used to solve this problem.
 - $\Rightarrow$ The idea is that once we get the solutions of Ω and Z . We can recover any $x_{i,j}$ that is not observed in $\mathcal{O}$ using $\hat{x}_{i,j} = \sum_p \Omega_{i,p} z_{j,p}$.
 - $\Rightarrow$ We can use this method when the missingness happens at random.

An iterative algorithm for matrix completion [2, 4]

Initialization:

- Create a complete data matrix $\tilde{\mathbf{X}}$ of dimension $n \times m$ containing the (i, j) element

$$\tilde{x}_{i,j} = \begin{cases} x_{i,j}, & \text{if } (i,j) \in \mathcal{O} \\ \bar{x}_j, & \text{if } (i,j) \notin \mathcal{O} \end{cases}$$

where $\bar{x}_j$ is the average of the observed values for the j th variable in the incomplete data matrix $\mathbf{X}$. Here, $\mathcal{O}$ denotes the set of observations. Any $(i,j) \notin \mathcal{O}$ are the missing values.

Repeat the following steps until the objective loss stops converge:

- Solve the following problem by computing the principal components $\mathbf{P}$ and $\mathbf{Z}$; then compute $\hat{\Omega}$ using the pseudo inverse ($\mathbf{P}$):

$$(\hat{\Omega}, \hat{\mathbf{Z}}) = \underset{\Omega \in \mathbb{R}^{n \times p}, \mathbf{Z} \in \mathbb{R}^{p \times m}}{\operatorname{argmin}} \sum_{j=1}^m \sum_{i=1}^n (\tilde{x}_{i,j} - \sum_p \Omega_{i,p} z_{j,p})^2$$

$\hat{\Omega} : \text{eigen-decomp}$
 $\hat{\mathbf{Z}} = \operatorname{inv}(\hat{\Omega}^T) \tilde{\mathbf{x}}$

- For each element $(i,j) \notin \mathcal{O}$, set $\tilde{x}_{i,j} \leftarrow \sum_p \hat{\Omega}_{i,p} \hat{z}_{j,p}$
- Compute the objective loss: $\sum_{(i,j) \in \mathcal{O}} (x_{i,j} - \sum_p \hat{\Omega}_{i,p} \hat{z}_{j,p})^2$ * loss = difference between observed data and

Output

- Return the estimated missing values: $\tilde{x}_{i,j}, (i,j) \notin \mathcal{O}$

A use case [2]

By applying the algorithm, we can predict the customer rating: $x_{i,j} = \sum_p \hat{\Omega}_{i,p} \hat{z}_{j,p}$. Furthermore we can interpret the p components in terms of *cliques* and *genres*.

- $\hat{\Omega}_{i,p}$ represents the strength with which the i th user belongs to the p th clique, where a clique is a group of customers that enjoys movies of the p th genre;
- $\hat{z}_{j,p}$ represents the strength with which the j th movie belongs to the p th genre.

* features.

sample

	Jerry Maguire	Oceans	Road to Perdition	A Fortunate Man	Catch Me If You Can	Driving Miss Daisy	The Two Popes	The Laundromat	Code 8	The Social Network	...
Customer 1	•	•	•	•	4	•	•	•	•	•	...
Customer 2	•	•	3	•	•	3	•	•	3	•	...
Customer 3	•	2	•	4	•	•	•	2	•	•	...
Customer 4	3	•	•	•	•	•	•	•	•	•	...
Customer 5	5	1	•	•	4	•	•	•	•	•	...
Customer 6	•	•	•	•	•	2	4	•	•	•	...
Customer 7	•	•	5	•	•	•	3	•	•	•	...
Customer 8	•	•	•	•	•	•	•	•	•	•	...
Customer 9	3	•	•	•	5	•	•	1	•	•	...
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮

TABLE 12.2. Excerpt of the Netflix movie rating data. The movies are rated from 1 (worst) to 5 (best). The symbol • represents a missing value: a movie that was not rated by the corresponding customer.

- 1 Table of contents
- 2 Supervised vs unsupervised learning
- 3 Principal component analysis (PCA)
 - The idea
 - What is a good projection matrix?
 - The solutions are the eigenvectors

The minimum-error formulation

- 4 PCA applications
 - data visualization (with codes)
 - incomplete data observation
- 5 **Appendix**

An example of finding eigenvectors in a matrix

Let $\mathbf{A}$ be an $n \times n$ matrix:

- 1 Find the eigenvalues λ of $\mathbf{A}$ by solving the equation $\det(\lambda \mathbf{I} - \mathbf{A}) = 0$
- 2 For each λ , find the basic eigenvectors $\mathbf{q} \neq 0$ by finding the basic solution to $(\lambda \mathbf{I} - \mathbf{A})\mathbf{q} = 0$

To verify your eigenvectors and eigenvalues, make sure that $\mathbf{A}\mathbf{q} = \lambda\mathbf{q}$.

Note that for PCA, $\mathbf{A} = \mathbf{X}\mathbf{X}^T$

Example: let $\mathbf{A} = \begin{bmatrix} -5 & 2 \\ -7 & 4 \end{bmatrix}$

1. Find the eigenvalues λ of $\mathbf{A}$ by solving $\det(\lambda \mathbf{I} - \mathbf{A}) = 0$

$$\det\left(\lambda \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} - \begin{bmatrix} -5 & 2 \\ -7 & 4 \end{bmatrix}\right) = 0$$

$$\det\left(\begin{bmatrix} \lambda + 5 & -2 \\ 7 & \lambda - 4 \end{bmatrix}\right) = 0$$

$$\lambda^2 + \lambda - 6 = 0$$

$$\lambda^2 + \lambda - 6 = (\lambda + 3)(\lambda - 2) = 0 \Rightarrow \lambda = -3, 2$$

2. For each λ , find the eigenvectors $\mathbf{q}_1$ and $\mathbf{q}_2$ by finding the basic solution to $(\lambda \mathbf{I} - \mathbf{A})\mathbf{q} = 0$ (next page)

PCA: example of finding eigenvectors in a matrix

Example: let $A = \begin{bmatrix} -5 & 2 \\ -7 & 4 \end{bmatrix}$

2. For each λ , find the first eigenvectors $\mathbf{q}_1$ by finding the basic solution to

$$(2 \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} - \begin{bmatrix} -5 & 2 \\ -7 & 4 \end{bmatrix})\mathbf{q}_1 = (\begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix} - \begin{bmatrix} -5 & 2 \\ -7 & 4 \end{bmatrix})\mathbf{q}_1 = \begin{bmatrix} 7 & -2 \\ 7 & -2 \end{bmatrix} \mathbf{q}_1 = 0$$

The basic eigenvector will be

$$\mathbf{q}_1 = \begin{bmatrix} 2 \\ 7 \end{bmatrix}$$

2.2 For $\lambda = -3$, find the basic eigenvectors $\mathbf{q}_2$ by finding the basic solution to

$$(-3 \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} - \begin{bmatrix} -5 & 2 \\ -7 & 4 \end{bmatrix})\mathbf{q}_2 = (\begin{bmatrix} -3 & 0 \\ 0 & -3 \end{bmatrix} - \begin{bmatrix} -5 & 2 \\ -7 & 4 \end{bmatrix})\mathbf{q}_2 = \begin{bmatrix} 2 & -2 \\ 7 & -7 \end{bmatrix} \mathbf{q}_2 = 0$$

The basic eigenvector will be

$$\mathbf{q}_2 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

For how to compute these eigenvectors please see [7.1 Eigenvalues and Eigenvectors of a Matrix](#)
Coding example: [PCA example](#).

- [1] Christopher M. Bishop. *Pattern Recognition and Machine Learning (Information Science and Statistics)*. 1st ed. Springer, 2007. ISBN: 0387310738.
- [2] Gareth James et al. *An Introduction to Statistical Learning: with Applications in R*. Springer, 2013. URL: https://hastie.su.domains/ISLP/ISLP_website.pdf.download.html.
- [3] *MNIST dataset*. <https://www.kaggle.com/datasets/hojjatk/mnist-dataset>. Accessed: 2026-02-28.
- [4] Mike Nguyen. *Foundations of Data Analysis*. Vol. 1. Springer Cham, 2025. DOI: [10.1007/978-3-032-01858-8](https://doi.org/10.1007/978-3-032-01858-8). URL: <https://doi.org/10.1007/978-3-032-01858-8>.